

Lab Report: Lab 5 - Implementing the Product Microservice

Contents

1. Cover Page	1
2. Abstract/Summary	1
3. Lab Specific Section: Implementing the Microservice.....	2
4. Conclusion & Reflection.....	3

1. Cover Page

- **Title:** Implementing the Product Microservice and Data Isolation for OmniDash
- **Course Info:** Software Architecture
- **Student Details:** Nguyễn Văn Trường, 23010371
- **Date:** May 10, 2026

2. Abstract/Summary

This lab details the practical implementation of the first fully independent microservice within the OmniDash ecosystem: the Product (or Core Resource) Service. The primary objective was to transition from theoretical decomposition to a tangible, autonomous application instance. We established a standalone web server utilizing the Flask framework, strictly configured to run on an isolated network port. Crucially, we enforced the "Database-per-Service" pattern by integrating SQLAlchemy as an Object-Relational Mapper (ORM) mapped to an isolated SQLite database, ensuring this service exclusively owns and manages its data domain. Finally, we exposed the defined RESTful Service Contracts to allow external querying and executed comprehensive isolation testing to verify the service's autonomy and fault tolerance.

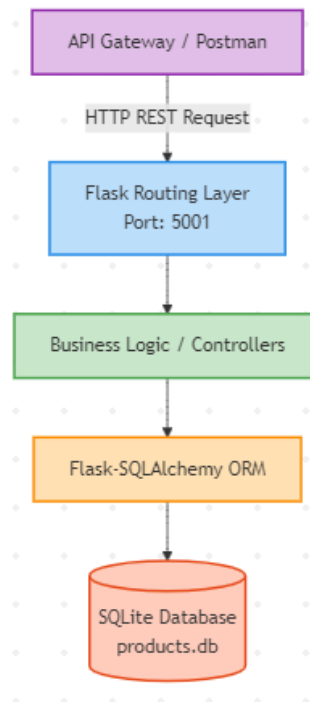
3. Lab Specific Section: Implementing the Microservice

This section documents the architectural logic and implementation strategies utilized to build the standalone Product Microservice.

3.1 Service Initialization and Data Isolation

To achieve true microservice autonomy, the service must operate independently from the main application monolith. In the OmniDash ecosystem, this service was initialized as a completely separate Flask application. To prevent port conflicts and establish clear network boundaries, the service was explicitly configured to listen on a dedicated port (e.g., Port 5001), differentiating it from the upcoming API Gateway and other background services.

A critical architectural milestone in this lab was the implementation of strict data isolation. We utilized a lightweight, file-based SQLite database (products.db) specifically provisioned for this service. By doing so, we mathematically guarantee that no other service in the OmniDash platform can perform cross-database SQL joins or directly read the product tables. All data access must strictly pass through the service's defined API boundaries.



3.2 Domain Modeling via Object-Relational Mapping (ORM)

Rather than writing raw, database-specific SQL queries, the system utilizes Flask-SQLAlchemy, a powerful Object-Relational Mapper. The domain entity—representing a core "Product" or "Resource" in OmniDash—is defined purely as a Class structure in the application code.

This entity encapsulates essential attributes such as a primary identifier, name, description, price, inventory stock, and an active status flag. The ORM automatically translates this Class definition into a physical database schema. When the application initializes, the ORM engine seamlessly creates the underlying tables if they do not exist. Furthermore, we implemented a data serialization method within the entity class itself, which systematically translates the complex Python object into a standardized, flat JSON dictionary. This ensures the Presentation Layer can easily format responses without knowing the underlying database column constraints.

4. Conclusion & Reflection

Lab 5 successfully translated microservice theory into a functional, deployed software component. By implementing a standalone Flask application bound to its own SQLite database via SQLAlchemy, we fully realized the "Database-per-Service" pattern. This ensures that the Product Service is highly cohesive and completely decoupled from the rest of the OmniDash ecosystem.

The successful isolation testing proves that this module can be independently scaled, deployed, and maintained without risking interference with other system components. This robust, self-contained architecture provides the exact foundation needed for the next phase of the project: routing and securing traffic via an API Gateway.